3. Any Turbo Stream declarations or server-side code needed to understand the interaction.

Pay particular attention to application-defined events such as:
`map:locate`
`map:reset`
`map:resize`
and any other custom events or cross-controller communication you find.
For this investigation:

1. Identify the main Stimulus controllers and what each one actually does.
2. Find where custom events are dispatched.
3. Find where those events are listened for.
4. Trace the complete event path from the initiating UI action to the receiving controller behavior.
5. Distinguish ordinary DOM events and Stimulus lifecycle behavior from application-defined communication.
6. Check whether one frontend controller knows directly about another controller or whether they communicate indirectly through events.
7. Record the important source paths and reliable line ranges.
8. Separate application-written JavaScript behavior from Stimulus, Turbo, Leaflet or browser-provided behavior.
9. Revisit the existing `Trip-Scoped Turbo Refresh Notifications` Observer candidate only if this investigation provides additional evidence. Do not automatically count it as a new instance.

Only after tracing the collaborations, identify any genuinely supported pattern candidates.
For each candidate:

1. Give the possible pattern and category.
2. State the actual participants.
3. Explain the mechanism that suggests the pattern.
4. Explain the strongest false-positive interpretation.
5. State whether the mechanism is application-owned, framework-provided or mixed.
6. Leave new candidates UNCONFIRMED.

Do not call something Observer simply because JavaScript events exist. Show the actual publisher/subscriber relationship first.
At the end give me:

1. Main frontend collaboration traces.
2. Custom-event producer/consumer table.
3. Participant and source-evidence table.
4. New pattern candidates only.
5. Whether the existing Turbo Observer candidate gains any stronger evidence.
6. Previously identified collaborations encountered again but not counted again.
7. Strongest non-pattern explanation.
8. Anything that still needs verification.

Worked for 1m 44s

DaPlanStan — Frontend Interaction Investigation

Source Investigation 06 · Original uploaded source ZIP · Static source analysis

New patterns unconfirmed

1. Initial source findings

I inspected the JavaScript files under `app/javascript/controllers/` and searched their corresponding views for controller attachments, event dispatches and listeners.

The directory contains **16 application-defined Stimulus controllers**, plus two Stimulus initialization/registration files.

The strongest cross-controller interaction is between three controllers:

- `TimelineController` publishes `map:locate` and `map:reset` when a user selects or deselects an itinerary row.
- `TripNavController` publishes `map:resize` when the itinerary panel becomes visible.
- `MapController` listens for all three events and performs the corresponding Leaflet operations.

This is a real, application-defined communication relationship. However, the event names are explicitly known to the producers and consumer, and the events are dispatched directly onto the map's DOM element. It should not automatically be classified as a fully decoupled Observer implementation.

Two other mechanisms are present but should be kept separate: normal DOM/Stimulus interactions within individual controllers, and server-driven Turbo Stream refreshes that may update the page and cause Stimulus controllers to reconnect.